

Lem-In: Ant Farm Pathfinding Simulation

Project Report

Course: Educational Practice

Astana IT University

Team Members:

Madina Sabyrgali | Tomiris Chekalin | Aida Mukhambetkaliyeva | Akerke Berdakul

June 2026

1. Team Members and Contributions

The project was developed by a team of four members, each assigned a distinct role to ensure structured collaboration and efficient delivery of all project components.

Role	Name	Responsibilities
Captain (Team Lead)	Madina Sabyrgali	Narrated the core algorithm modules in the demo: node-splitting and Edmonds-Karp max-flow pathfinding (bfs.go), greedy ant distribution and turn-by-turn simulation (solver.go), plan.md breakdown, and the live demo showing valid runs, thousand-ant performance, error handling, and the visualizer.
Demo Video Creator	Tomiris Chekalin	Edited the demo video walkthrough covering: project introduction, .gitignore and go.mod setup, README usage instructions, examples folder structure (8 valid + 2 bad input files), internal/app/app.go pipeline orchestration, and internal/model/input.go domain types.
Tester	Aida Mukhambetkaliyeva	Writing and executing tests, verifying edge cases and error handling. Developed the bonus visualizer: reads lem-in output, draws rooms and tunnels using Bresenham's line algorithm, labels start/end rooms, shows live ant counts per room, and auto-adjusts canvas height to prevent label overlap.
Reporter & Analyst	Akerke Berdakul	Writing the project report, documenting methodology and results. Narrated parser and validation modules in the demo: line-by-line parsing logic, ant count validation, ###start/###end marker handling, self-link and duplicate link detection, and post-parse integrity checks for duplicate room names and undefined rooms in links.

2. Project Overview

Lem-In is a command-line simulation program written in the Go programming language. The program models an ant colony navigation problem: given a map of rooms connected by tunnels (links), and a specified number of ants starting at a designated start room, the program must move all ants to a designated end room in as few turns as possible. Each turn, ants may move simultaneously, but at most one ant can occupy a non-terminal room at any given time. The program reads input from a text file, parses the graph structure, validates it, finds optimal vertex-disjoint paths using a max-flow algorithm, distributes ants across those paths intelligently, and simulates the movement step by step, printing the result to standard output.

The project was built using Go standard library packages, demonstrating proficiency in graph algorithms, file I/O, input validation, and clean modular architecture. It also includes an optional visualizer tool that renders the ant movement graphically in a terminal window.

3. Problem Statement

The main challenge of the Lem-In project is to find the fastest way to move all ants from the start room to the end room in a graph while ensuring that no two ants occupy the same intermediate room at the same time.

The program must also detect and reject invalid maps, such as missing start or end rooms, duplicate room names, invalid ant counts, self-links, links to nonexistent rooms, or maps with no valid path. In these cases, it should display an error message in the format: **ERROR: invalid data format, <reason>**.

To minimize the number of turns, the solution should identify multiple non-overlapping paths and distribute ants efficiently among them. This avoids congestion on a single path and improves overall performance.

4. Methodology / Tools and Technologies Used

The project was developed in **Go 1.24.2** using only the standard library, providing efficient performance, strong concurrency support, and clean system-level programming capabilities. Development followed a phased approach documented in *plan.md*: input parsing and validation, path discovery, and ant simulation. This structure enabled parallel development and smooth integration of components.

Path finding was implemented using **node-splitting** and the **Edmonds-Karp** maximum-flow algorithm. Node splitting converts each room into input and output nodes connected by a capacity-1 edge, ensuring that intermediate rooms cannot be occupied by multiple ants simultaneously and allowing the discovery of the maximum number of vertex-disjoint paths.

Version control was managed with **Git**, and the project follows Go conventions with a **cmd/** directory for entry points and an **internal/** directory for core packages.

5. Main Features / Model Description

The program follows a layered architecture with clearly separated responsibilities. The **model layer** defines the core data structures, including the ant farm configuration, room information, and graph connections.

The **parser layer** reads the input file, processes the ant count, start and end room markers, room definitions, and links between rooms. It also detects invalid inputs such as duplicate markers, self-links, and duplicate connections.

The **validation layer** performs semantic checks, ensuring that room names are unique, all referenced rooms exist, and the command-line arguments are valid.

The **pathfinding layer** uses a node-split graph and the **Edmonds-Karp maximum-flow algorithm** to find all vertex-disjoint paths between the start and end rooms. Intermediate rooms have a capacity of one, preventing multiple ants from occupying the same room simultaneously.

The **solver layer** distributes ants across the available paths using a greedy strategy that balances path length and current load. Ant movements are then simulated turn by turn, minimizing the total number of turns required to reach the destination.

6. Implementation Explanation

The program starts in **cmd/main.go**, which processes command-line arguments and calls `app.Run()`. The **app layer** coordinates the entire workflow: argument validation, parsing, input validation, path finding, solving, and output generation. Any failure produces an error message prefixed with **ERROR: invalid data format**.

The **parser** validates the ant count, ensures the start and end markers appear exactly once, checks room definitions, rejects reserved room name prefixes (``#`` and ``L``), and detects duplicate or invalid links.

The **pathfinding module** builds a residual graph and applies the **Edmonds-Karp algorithm** to find augmenting paths. Once the maximum flow is reached, it reconstructs all vertex-disjoint paths from the resulting flow network.

The **solver** distributes ants using the cost function ``path length + assigned ants``, balancing the load across paths. It then simulates ant movement turn by turn, outputting moves in the format ``L<id>-<room>``.

An optional **visualizer** provides a terminal-based graphical view of the ant farm and ant movements using the room coordinates from the input file.

7. Screenshots

```

Gantt-2023-10-26 15:30:00
1431:net:1.222:10000 1.431:10000-1.432:10000 10000:10000
1432:net:1.222:10000 1.432:10000-1.433:10000 10000:10000
1433:net:1.222:10000 1.433:10000-1.434:10000 10000:10000
1434:net:1.222:10000 1.434:10000-1.435:10000 10000:10000
1435:net:1.222:10000 1.435:10000-1.436:10000 10000:10000
1436:net:1.222:10000 1.436:10000-1.437:10000 10000:10000
1437:net:1.222:10000 1.437:10000-1.438:10000 10000:10000
1438:net:1.222:10000 1.438:10000-1.439:10000 10000:10000
1439:net:1.222:10000 1.439:10000-1.440:10000 10000:10000
1440:net:1.222:10000 1.440:10000-1.441:10000 10000:10000
1441:net:1.222:10000 1.441:10000-1.442:10000 10000:10000
1442:net:1.222:10000 1.442:10000-1.443:10000 10000:10000
1443:net:1.222:10000 1.443:10000-1.444:10000 10000:10000
1444:net:1.222:10000 1.444:10000-1.445:10000 10000:10000
1445:net:1.222:10000 1.445:10000-1.446:10000 10000:10000
1446:net:1.222:10000 1.446:10000-1.447:10000 10000:10000
1447:net:1.222:10000 1.447:10000-1.448:10000 10000:10000
1448:net:1.222:10000 1.448:10000-1.449:10000 10000:10000
1449:net:1.222:10000 1.449:10000-1.450:10000 10000:10000
1450:net:1.222:10000 1.450:10000-1.451:10000 10000:10000
1451:net:1.222:10000 1.451:10000-1.452:10000 10000:10000
1452:net:1.222:10000 1.452:10000-1.453:10000 10000:10000
1453:net:1.222:10000 1.453:10000-1.454:10000 10000:10000
1454:net:1.222:10000 1.454:10000-1.455:10000 10000:10000
1455:net:1.222:10000 1.455:10000-1.456:10000 10000:10000
1456:net:1.222:10000 1.456:10000-1.457:10000 10000:10000
1457:net:1.222:10000 1.457:10000-1.458:10000 10000:10000
1458:net:1.222:10000 1.458:10000-1.459:10000 10000:10000
1459:net:1.222:10000 1.459:10000-1.460:10000 10000:10000
1460:net:1.222:10000 1.460:10000-1.461:10000 10000:10000
1461:net:1.222:10000 1.461:10000-1.462:10000 10000:10000
1462:net:1.222:10000 1.462:10000-1.463:10000 10000:10000
1463:net:1.222:10000 1.463:10000-1.464:10000 10000:10000
1464:net:1.222:10000 1.464:10000-1.465:10000 10000:10000
1465:net:1.222:10000 1.465:10000-1.466:10000 10000:10000
1466:net:1.222:10000 1.466:10000-1.467:10000 10000:10000
1467:net:1.222:10000 1.467:10000-1.468:10000 10000:10000
1468:net:1.222:10000 1.468:10000-1.469:10000 10000:10000
1469:net:1.222:10000 1.469:10000-1.470:10000 10000:10000
1470:net:1.222:10000 1.470:10000-1.471:10000 10000:10000
1471:net:1.222:10000 1.471:10000-1.472:10000 10000:10000
1472:net:1.222:10000 1.472:10000-1.473:10000 10000:10000
1473:net:1.222:10000 1.473:10000-1.474:10000 10000:10000
1474:net:1.222:10000 1.474:10000-1.475:10000 10000:10000
1475:net:1.222:10000 1.475:10000-1.476:10000 10000:10000
1476:net:1.222:10000 1.476:10000-1.477:10000 10000:10000
1477:net:1.222:10000 1.477:10000-1.478:10000 10000:10000
1478:net:1.222:10000 1.478:10000-1.479:10000 10000:10000
1479:net:1.222:10000 1.479:10000-1.480:10000 10000:10000
1480:net:1.222:10000 1.480:10000-1.481:10000 10000:10000
1481:net:1.222:10000 1.481:10000-1.482:10000 10000:10000
1482:net:1.222:10000 1.482:10000-1.483:10000 10000:10000
1483:net:1.222:10000 1.483:10000-1.484:10000 10000:10000
1484:net:1.222:10000 1.484:10000-1.485:10000 10000:10000
1485:net:1.222:10000 1.485:10000-1.486:10000 10000:10000
1486:net:1.222:10000 1.486:10000-1.487:10000 10000:10000
1487:net:1.222:10000 1.487:10000-1.488:10000 10000:10000
1488:net:1.222:10000 1.488:10000-1.489:10000 10000:10000
1489:net:1.222:10000 1.489:10000-1.490:10000 10000:10000
1490:net:1.222:10000 1.490:10000-1.491:10000 10000:10000
1491:net:1.222:10000 1.491:10000-1.492:10000 10000:10000
1492:net:1.222:10000 1.492:10000-1.493:10000 10000:10000
1493:net:1.222:10000 1.493:10000-1.494:10000 10000:10000
1494:net:1.222:10000 1.494:10000-1.495:10000 10000:10000
1495:net:1.222:10000 1.495:10000-1.496:10000 10000:10000
1496:net:1.222:10000 1.496:10000-1.497:10000 10000:10000
1497:net:1.222:10000 1.497:10000-1.498:10000 10000:10000
1498:net:1.222:10000 1.498:10000-1.499:10000 10000:10000
1499:net:1.222:10000 1.499:10000-1.500:10000 10000:10000
Gantt-2023-10-26 15:30:00

```

[illegible]

Example 5 with
visualizer

Regular
examples(example00.txt
-example07.txt)

```

L1-1501-peter
* GuestC2-65 ien-1n % go run cmd/main.go examples/example07.txt

1000
##$start
richard 8 6
gilfoyle 6 3
erlich 9 6
dinh 6 9
jimYoung 11 7
$wand
peter 14 6
richard-dinh
dinh-jimYoung
richard-gilfoyle
gilfoyle-peter
gilfoyle-erlich
richard-erlich
erlich-jimYoung
jimYoung-peter

L1-dinh L501-gilfoyle
L1-jimYoung L2-dinh L501-peter L502-gilfoyle
L1-peter L2-jimYoung L3-dinh L502-peter L503-gilfoyle
L2-peter L3-jimYoung L4-dinh L503-peter L504-gilfoyle
L3-peter L4-jimYoung L5-dinh L504-peter L505-gilfoyle
L4-peter L5-jimYoung L6-dinh L505-peter L506-gilfoyle
L5-peter L6-jimYoung L7-dinh L506-peter L507-gilfoyle
L6-peter L7-jimYoung L8-dinh L507-peter L508-gilfoyle
L7-peter L8-jimYoung L9-dinh L508-peter L509-gilfoyle
L8-peter L9-jimYoung L10-dinh L509-peter L510-gilfoyle
L9-peter L10-jimYoung L11-dinh L510-peter L511-gilfoyle
L10-peter L11-jimYoung L12-dinh L511-peter L512-gilfoyle
L11-peter L12-jimYoung L13-dinh L512-peter L513-gilfoyle
L12-peter L13-jimYoung L14-dinh L513-peter L514-gilfoyle
L13-peter L14-jimYoung L15-dinh L514-peter L515-gilfoyle
L14-peter L15-jimYoung L16-dinh L515-peter L516-gilfoyle
L15-peter L16-jimYoung L17-dinh L516-peter L517-gilfoyle
L16-peter L17-jimYoung L18-dinh L517-peter L518-gilfoyle
L17-peter L18-jimYoung L19-dinh L518-peter L519-gilfoyle
L18-peter L19-jimYoung L20-dinh L519-peter L520-gilfoyle
L19-peter L20-jimYoung L21-dinh L520-peter L521-gilfoyle
L20-peter L21-jimYoung L22-dinh L521-peter L522-gilfoyle
L21-peter L22-jimYoung L23-dinh L522-peter L523-gilfoyle
L22-peter L23-jimYoung L24-dinh L523-peter L524-gilfoyle
L23-peter L24-jimYoung L25-dinh L524-peter L525-gilfoyle
L24-peter L25-jimYoung L26-dinh L525-peter L526-gilfoyle
L25-peter L26-jimYoung L27-dinh L526-peter L527-gilfoyle
L26-peter L27-jimYoung L28-dinh L527-peter L528-gilfoyle
L27-peter L28-jimYoung L29-dinh L528-peter L529-gilfoyle
L28-peter L29-jimYoung L30-dinh L529-peter L530-gilfoyle
L29-peter L30-jimYoung L31-dinh L530-peter L531-gilfoyle
L30-peter L31-jimYoung L32-dinh L531-peter L532-gilfoyle
L31-peter L32-jimYoung L33-dinh L532-peter L533-gilfoyle
L32-peter L33-jimYoung L34-dinh L533-peter L534-gilfoyle
L33-peter L34-jimYoung L35-dinh L534-peter L535-gilfoyle
L34-peter L35-jimYoung L36-dinh L535-peter L536-gilfoyle
L35-peter L36-jimYoung L37-dinh L536-peter L537-gilfoyle
L36-peter L37-jimYoung L38-dinh L537-peter L538-gilfoyle
L37-peter L38-jimYoung L39-dinh L538-peter L539-gilfoyle
L38-peter L39-jimYoung L40-dinh L539-peter L540-gilfoyle
L39-peter L40-jimYoung L41-dinh L540-peter L541-gilfoyle
L40-peter L41-jimYoung L42-dinh L541-peter L542-gilfoyle
L41-peter L42-jimYoung L43-dinh L542-peter L543-gilfoyle
L42-peter L43-jimYoung L44-dinh L543-peter L544-gilfoyle
L43-peter L44-jimYoung L45-dinh L544-peter L545-gilfoyle
L44-peter L45-jimYoung L46-dinh L545-peter L546-gilfoyle
L45-peter L46-jimYoung L47-dinh L546-peter L547-gilfoyle
L46-peter L47-jimYoung L48-dinh L547-peter L548-gilfoyle
L47-peter L48-jimYoung L49-dinh L548-peter L549-gilfoyle
L48-peter L49-jimYoung L50-dinh L549-peter L550-gilfoyle
L49-peter L50-jimYoung L51-dinh L550-peter L551-gilfoyle

```

[illegible]

Examples with wrong data
(badexample00.txt,badexample01.txt)

```

13-@rtr 14-jmcmg 15-perc 16-gilfoyle
16-@rtr 15-perc
16-@G06102-05 16m-jn 4 go run cmd/mis-go examples/example05.tst

9
9mme
9merrt
start
9merr 1
9merr 2
CP 1 8
C1 2 6
C2 3 6
C3 4 6
14 5 6
15 6 6
16 6 6
Ad 1 2
Ad 2 2
Ad 3 1
Ad 4 1
Ad 5 1
Ad 6 1
Ad 7 1
Ad 8 1
Ad 9 1
Ad 10 1
Ad 11 1
Ad 12 1
Ad 13 1
Ad 14 1
Ad 15 1
Ad 16 1
Ad 17 1
Ad 18 1
Ad 19 1
Ad 20 1
Ad 21 1
Ad 22 1
Ad 23 1
Ad 24 1
Ad 25 1
Ad 26 1
Ad 27 1
Ad 28 1
Ad 29 1
Ad 30 1
Ad 31 1
Ad 32 1
Ad 33 1
Ad 34 1
Ad 35 1
Ad 36 1
Ad 37 1
Ad 38 1
Ad 39 1
Ad 40 1
Ad 41 1
Ad 42 1
Ad 43 1
Ad 44 1
Ad 45 1
Ad 46 1
Ad 47 1
Ad 48 1
Ad 49 1
Ad 50 1
Ad 51 1
Ad 52 1
Ad 53 1
Ad 54 1
Ad 55 1
Ad 56 1
Ad 57 1
Ad 58 1
Ad 59 1
Ad 60 1
Ad 61 1
Ad 62 1
Ad 63 1
Ad 64 1
Ad 65 1
Ad 66 1
Ad 67 1
Ad 68 1
Ad 69 1
Ad 70 1
Ad 71 1
Ad 72 1
Ad 73 1
Ad 74 1
Ad 75 1
Ad 76 1
Ad 77 1
Ad 78 1
Ad 79 1
Ad 80 1
Ad 81 1
Ad 82 1
Ad 83 1
Ad 84 1
Ad 85 1
Ad 86 1
Ad 87 1
Ad 88 1
Ad 89 1
Ad 90 1
Ad 91 1
Ad 92 1
Ad 93 1
Ad 94 1
Ad 95 1
Ad 96 1
Ad 97 1
Ad 98 1
Ad 99 1
Ad 100 1
Ad 101 1
Ad 102 1
Ad 103 1
Ad 104 1
Ad 105 1
Ad 106 1
Ad 107 1
Ad 108 1
Ad 109 1
Ad 110 1
Ad 111 1
Ad 112 1
Ad 113 1
Ad 114 1
Ad 115 1
Ad 116 1
Ad 117 1
Ad 118 1
Ad 119 1
Ad 120 1
Ad 121 1
Ad 122 1
Ad 123 1
Ad 124 1
Ad 125 1
Ad 126 1
Ad 127 1
Ad 128 1
Ad 129 1
Ad 130 1
Ad 131 1
Ad 132 1
Ad 133 1
Ad 134 1
Ad 135 1
Ad 136 1
Ad 137 1
Ad 138 1
Ad 139 1
Ad 140 1
Ad 141 1
Ad 142 1
Ad 143 1
Ad 144 1
Ad 145 1
Ad 146 1
Ad 147 1
Ad 148 1
Ad 149 1
Ad 150 1
Ad 151 1
Ad 152 1
Ad 153 1
Ad 154 1
Ad 155 1
Ad 156 1
Ad 157 1
Ad 158 1
Ad 159 1
Ad 160 1
Ad 161 1
Ad 162 1
Ad 163 1
Ad 164 1
Ad 165 1
Ad 166 1
Ad 167 1
Ad 168 1
Ad 169 1
Ad 170 1
Ad 171 1
Ad 172 1
Ad 173 1
Ad 174 1
Ad 175 1
Ad 176 1
Ad 177 1
Ad 178 1
Ad 179 1
Ad 180 1
Ad 181 1
Ad 182 1
Ad 183 1
Ad 184 1
Ad 185 1
Ad 186 1
Ad 187 1
Ad 188 1
Ad 189 1
Ad 190 1
Ad 191 1
Ad 192 1
Ad 193 1
Ad 194 1
Ad 195 1
Ad 196 1
Ad 197 1
Ad 198 1
Ad 199 1
Ad 200 1
Ad 201 1
Ad 202 1
Ad 203 1
Ad 204 1
Ad 205 1
Ad 206 1
Ad 207 1
Ad 208 1
Ad 209 1
Ad 210 1
Ad 211 1
Ad 212 1
Ad 213 1
Ad 214 1
Ad 215 1
Ad 216 1
Ad 217 1
Ad 218 1
Ad 219 1
Ad 220 1
Ad 221 1
Ad 222 1
Ad 223 1
Ad 224 1
Ad 225 1
Ad 226 1
Ad 227 1
Ad 228 1
Ad 229 1
Ad 230 1
Ad 231 1
Ad 232 1
Ad 233 1
Ad 234 1
Ad 235 1
Ad 236 1
Ad 237 1
Ad 238 1
Ad 239 1
Ad 240 1
Ad 241 1
Ad 242 1
Ad 243 1
Ad 244 1
Ad 245 1
Ad 246 1
Ad 247 1
Ad 248 1
Ad 249 1
Ad 250 1
Ad 251 1
Ad 252 1
Ad 253 1
Ad 254 1
Ad 255 1
Ad 256 1
Ad 257 1
Ad 258 1
Ad 259 1
Ad 260 1
Ad 261 1
Ad 262 1
Ad 263 1
Ad 264 1
Ad 265 1
Ad 266 1
Ad 267 1
Ad 268 1
Ad 269 1
Ad 270 1
Ad 271 1
Ad 272 1
Ad 273 1
Ad 274 1
Ad 275 1
Ad 276 1
Ad 277 1
Ad 278 1
Ad 279 1
Ad 280 1
Ad 281 1
Ad 282 1
Ad 283 1
Ad 284 1
Ad 285 1
Ad 286 1
Ad 287 1
Ad 288 1
Ad 289 1
Ad 290 1
Ad 291 1
Ad 292 1
Ad 293 1
Ad 294 1
Ad 295 1
Ad 296 1
Ad 297 1
Ad 298 1
Ad 299 1
Ad 300 1
Ad 301 1
Ad 302 1
Ad 303 1
Ad 304 1
Ad 305 1
Ad 306 1
Ad 307 1
Ad 308 1
Ad 309 1
Ad 310 1
Ad 311 1
Ad 312 1
Ad 313 1
Ad 314 1
Ad 315 1
Ad 316 1
Ad 317 1
Ad 318 1
Ad 319 1
Ad 320 1
Ad 321 1
Ad 322 1
Ad 323 1
Ad 324 1
Ad 325 1
Ad 326 1
Ad 327 1
Ad 328 1
Ad 329 1
Ad 330 1
Ad 331 1
Ad 332 1
Ad 333 1
Ad 334 1
Ad 335 1
Ad 336 1
Ad 337 1
Ad 338 1
Ad 339 1
Ad 340 1
Ad 341 1
Ad 342 1
Ad 343 1
Ad 344 1
Ad 345 1
Ad 346 1
Ad 347 1
Ad 348 1
Ad 349 1
Ad 350 1
Ad 351 1
Ad 352 1
Ad 353 1
Ad 354 1
Ad 355 1
Ad 356 1
Ad 357 1
Ad 358 1
Ad 359 1
Ad 360 1
Ad 361 1
Ad 362 1
Ad 363 1
Ad 364 1
Ad 365 1
Ad 366 1
Ad 367 1
Ad 368 1
Ad 369 1
Ad 370 1
Ad 371 1
Ad 372 1
Ad 373 1
Ad 374 1
Ad 375 1
Ad 376 1
Ad 377 1
Ad 378 1
Ad 379 1
Ad 380 1
Ad 381 1
Ad 382 1
Ad 383 1
Ad 384 1
Ad 385 1
Ad 386 1
Ad 387 1
Ad 388 1
Ad 389 1
Ad 390 1
Ad 391 1
Ad 392 1
Ad 393 1
Ad 394 1
Ad 395 1
Ad 396 1
Ad 397 1
Ad 398 1
Ad 399 1
Ad 400 1
Ad 401 1
Ad 402 1
Ad 403 1
Ad 404 1
Ad 405 1
Ad 406 1
Ad 407 1
Ad 408 1
Ad 409 1
Ad 410 1
Ad 411 1
Ad 412 1
Ad 413 1
Ad 414 1
Ad 415 1
Ad 416 1
Ad 417 1
Ad 418 1
Ad 419 1
Ad 420 1
Ad 421 1
Ad 422 1
Ad 423 1
Ad 424 1
Ad 425 1
Ad 426 1
Ad 427 1
Ad 428 1
Ad 429 1
Ad 430 1
Ad 431 1
Ad 432 1
Ad 433 1
Ad 434 1
Ad 435 1
Ad 436 1
Ad 437 1
Ad 438 1
Ad 439 1
Ad 440 1
Ad 441 1
Ad 442 1
Ad 443 1
Ad 444 1
Ad 445 1
Ad 446 1
Ad 447 1
Ad 448 1
Ad 449 1
Ad 450 1
Ad 451 1
Ad 452 1
Ad 453 1
Ad 454 1
Ad 455 1
Ad 456 1
Ad 457 1
Ad 458 1
Ad 459 1
Ad 460 1
Ad 461 1
Ad 462 1
Ad 463 1
Ad 464 1
Ad 465 1
Ad 466 1
Ad 467 1
Ad 468 1
Ad 469 1
Ad 470 1
Ad 471 1
Ad 472 1
Ad 473 1
Ad 474 1
Ad 475 1
Ad 476 1
Ad 477 1
Ad 478 1
Ad 479 1
Ad 480 1
Ad 481 1
Ad 482 1
Ad 483 1
Ad 484 1
Ad 485 1
Ad 486 1
Ad 487 1
Ad 488 1
Ad 489 1
Ad 490 1
Ad 491 1
Ad 492 1
Ad 493 1
Ad 494 1
Ad 495 1
Ad 496 1
Ad 497 1
Ad 498 1
Ad 499 1
Ad 500 1
Ad 501 1
Ad 502 1
Ad 503 1
Ad 504 1
Ad 505 1
Ad 506 1
Ad 507 1

```

```
L4-5
● Guest@C2-05 lem-in % go run cmd/main.go examples/example04.txt

9
##start
richard 0 6
gilfoyle 6 3
erlich 9 6
dinish 6 9
jimYoung 11 7
##end
peter 14 6
richard-dinish
dinish-jimYoung
richard-gilfoyle
gilfoyle-peter
gilfoyle-erlich
richard-erlich
erlich-jimYoung
jimYoung-peter

L1-dinish L5-gilfoyle
L1-jimYoung L2-dinish L5-peter L6-gilfoyle
L1-peter L2-jimYoung L3-dinish L6-peter L7-gilfoyle
L2-peter L3-jimYoung L4-dinish L7-peter L8-gilfoyle
L3-peter L4-jimYoung L8-peter L9-gilfoyle
L4-peter L9-peter
○ Guest@C2-05 lem-in %
```

```
L9-3 L28-3
● Guest@C2-05 lem-in % go run cmd/main.go examples/example03.txt
4
4 5 4
##start
0 1 4
1 3 6
##end
5 6 4
2 3 4
3 3 1
0-1
2-4
1-4
0-2
4-5
3-0
4-3

L1-1
L1-4 L2-1
L1-5 L2-4 L3-1
L2-5 L3-4 L4-1
L3-5 L4-4
L4-5
○ Guest@C2-05 lem-in %
```

```

L4-end
• Guest@C2-05 lem-in % go run cmd/main.go examples/example02.txt

20
##start
0 2 0
1 4 1
2 6 0
##end
3 5 3
0-1
0-3
1-2
3-2

L1-1 L10-3
L1-2 L2-1 L11-3
L1-3 L2-2 L3-1 L12-3
L2-3 L3-2 L4-1 L13-3
L3-3 L4-2 L5-1 L14-3
L4-3 L5-2 L6-1 L15-3
L5-3 L6-2 L7-1 L16-3
L6-3 L7-2 L8-1 L17-3
L7-3 L8-2 L9-1 L18-3
L8-3 L9-2 L19-3
L9-3 L20-3

• Guest@C2-05 lem-in %

```

```

L4-1
Guest@C2-05 lem-in % go run cmd/main.go examples/example01.txt

10
##start
start 1 6
0 4 8
0 6 8
n 6 6
e 8 4
t 1 9
E 5 9
a 8 9
n 8 6
h 4 6
A 5 2
c 8 1
k 11 2
##end
end 11 6
start-t
n-e
a-m
A-c
0-o
E-a
k-end
start-h
o-n
m-end
t-e
start-0
h-A
e-end
C-k
n-m
h-n

L1-t L5-h L8-0
L1-E L2-t L5-A L6-h L8-o L9-0
L1-a L2-E L3-t L5-c L6-A L7-h L8-n L9-o L10-0
L1-m L2-a L3-E L4-t L5-k L6-c L7-A L8-e L9-n L10-o
L1-end L2-m L3-a L4-E L5-end L6-k L7-c L8-end L9-e L10-n
L2-end L3-m L4-a L6-end L7-k L9-end L10-e
L3-end L4-m L7-end L10-end
L4-end
Guest@C2-05 lem-in %

```

```

L4-end L5-A2 L6-end L7-15 L8-end L9-end
L5-end L7-end
Guest@C2-05 lem-in % go run cmd/main.go examples/example00.txt

4
##start
0 0 3
2 2 5
3 4 0
##end
1 8 3
0-2
2-3
3-1

L1-2
L1-3 L2-2
L1-1 L2-3 L3-2
L2-1 L3-3 L4-2
L3-1 L4-3
L4-1
Guest@C2-05 lem-in %

```


8. Testing or Evaluation Results

```
ok      01.tomorrow-school.ai/git/msabyrga/lem-in.git/internal/parser  0.601s
Guest@C2-05 lem-in % go test ./internal/validation/ -v
=== RUN   TestArgsEmpty
--- PASS: TestArgsEmpty (0.00s)
=== RUN   TestArgsTooMany
--- PASS: TestArgsTooMany (0.00s)
=== RUN   TestArgsValid
--- PASS: TestArgsValid (0.00s)
=== RUN   TestInputDuplicatedRoom
--- PASS: TestInputDuplicatedRoom (0.00s)
=== RUN   TestInputUnknownLink
--- PASS: TestInputUnknownLink (0.00s)
PASS
ok      01.tomorrow-school.ai/git/msabyrga/lem-in.git/internal/validation0.330s
```

Unit test results from internal/validation/validation_test.go, covering duplicate room detection, unknown link detection, and edge cases: All 5 tests passed: empty args, too many args, valid args, duplicated room, unknown link .ok ... 0.330s = finished in 0.330 seconds

```
Guest@C2-05 lem-in % go test ./internal/parser/ -v
=== RUN   TestParseValidInput
--- PASS: TestParseValidInput (0.00s)
=== RUN   TestParseZeroAnts
--- PASS: TestParseZeroAnts (0.00s)
=== RUN   TestParseMissingStart
--- PASS: TestParseMissingStart (0.00s)
=== RUN   TestParseMissingEnd
--- PASS: TestParseMissingEnd (0.00s)
=== RUN   TestParseDuplicatedLink
--- PASS: TestParseDuplicatedLink (0.00s)
PASS
ok      01.tomorrow-school.ai/git/msabyrga/lem-in.git/internal/parser  0.601s
Guest@C2-05 lem-in % go test ./internal/validation/ -v
```

. Unit test results from internal/parser/parser_test.go, covering valid and invalid input parsing scenarios.

- --- PASS: TestParseValidInput (0.00s) means it passed in 0 seconds (very fast)
- All 5 tests passed: valid input, zero ants, missing start, missing end, duplicated link
- PASS at the bottom = the whole package passed

ok ... 0.601s = total time for the package was 0.601 seconds

9. Difficulties and Solutions

A major challenge was implementing node-splitting for the Edmonds-Karp max-flow algorithm, since the problem constrains rooms (nodes) rather than edges. This required splitting each room into room_in and room_out with a unit-capacity edge, which took significant debugging to implement correctly.

Another difficulty was reconstructing paths from the residual graph, as Edmonds-Karp does not explicitly store them. This was solved by tracing edges whose capacities were reduced during flow and rebuilding paths from start to end while marking used edges.

The ant distribution and simulation logic was also complex. To avoid collisions, ants were assigned delays based on their position in a path queue, ensuring one-ant-per-turn entry into each path.

Finally, input validation edge cases (self-links, duplicate rooms, invalid prefixes, missing rooms) required careful incremental handling and extensive testing.

10. Conclusion

The Lem-In project demonstrates a graph optimization solution in Go using Edmonds-Karp max-flow on a node-split graph to find vertex-disjoint paths and minimize ant movement turns. It reinforced key concepts such as graph modeling, BFS-based pathfinding, flow networks, and greedy optimization, while the modular architecture improved maintainability and testing. The project also provided experience with Go project structure, standard library usage, and translating algorithms into working systems. The phased development approach improved organization and parallel work.

Future improvements could include more deterministic path sorting, a formal collision verifier, and an enhanced visualizer with better tracking features.

11. GitHub and Demo Video Links

GitHub Repository:

[aitu-educational-practice-2026/educational-practice-project-nomenzone:](https://github.com/qzxcvwyil/lem-in)

[aitu-educational-practice-2026-classroom-db4064-educational-practice-project-education](https://github.com/qzxcvwyil/lem-in)

[al-practice-te created by GitHub Classroom](https://github.com/qzxcvwyil/lem-in)

To clone and run the project:

1. Clone project: ``git clone https://github.com/qzxcvwyil/lem-in.git``
2. Open directory: ``cd lem-in``
3. Start the app: ``go run cmd/main.go examples/example00.txt``

Demo Video:

<https://youtu.be/b7wg2FZxa1E?si=eGh0K8FyY2PCvVAC>